



slington college
(इरिलिङ्टन कलेज)

Module Code & Module Title
Level 6 – Data and Web Development

Assessment Type
Semester 2025/26 Autumn

Student Name: Simanka Paudel

London Met ID: 23050271

College ID: NP01CP4A230098

Assignment Due Date: 24/12/2025.

Assignment Submission Date: 24/12/2025

Submitted To: Ms Shresha Rajbhandari

Word Count: 771

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submissions and a mark of zero will be awarded.

Table of Contents

1	Introduction	1
2	Data Modelling and Normalization	2
2.1	Normalization	2
2.2	Initial ERD Creation.....	4
2.3	Final ERD Using Data Modeler	4
2.4	Database implementation	5
2.4.1	Table Creation	5
2.4.2	Table Insertion.....	12

Table of figures

Figure 1: Initial ERD diagram	4
Figure 2: Final ERD using Data Modeler	5
Figure 3: 'User' table creation query	5
Figure 4: 'Movie' table creation query	5
Figure 5: 'Theatre' table creation query	6
Figure 6: Theatre table altered	6
Figure 7: Theatre table altered	6
Figure 8: 'Hall' table creation query	6
Figure 9: 'Show' table creation query query	6
Figure 10: 'Ticket' table creation query	7
Figure 11: 'User_Movie' table creation query	7
Figure 12: 'Theater_Movie_User' table creation	8
Figure 13: 'Theater_Movie_User' table alteration query	8
Figure 14: 'HTMU' table creation and alteration query	9
Figure 15: 'HTMU' table alteration query	9
Figure 16: 'SHTMU' table creation query	10
Figure 17: 'SHTMU' table alteration query	10
Figure 18: 'SHTMU' table alteration query	11
Figure 19: 'TSHTMU' table creation query	11
Figure 20: 'TSHTMU' table alteration query	12
Figure 21: Inserting data into 'USERS' table	12
Figure 22: Inserting data into 'USERS' table	13
Figure 23: Output of 'USERS' table	13
Figure 24: Insertion of data in 'Movie' table	13
Figure 25: Update on 'Movie' table	14
Figure 26: Output of Movie table	14
Figure 27: Insertion of data in 'Theatre' table	15
Figure 28: Output of Theatre table	15
Figure 29: Insertion of data in 'Hall' table	16
Figure 30: Output of 'Hall' table	16
Figure 31: Inserting data into 'Show' table	16
Figure 32: Inserting of data in Show table	17
Figure 33: Output of 'Show' table	17
Figure 34: Inserting of data in Ticket table	18
Figure 35: Output of 'Ticket' table	18
Figure 36: Inserting data into 'User_Movie' table	19
Figure 37: Output of User_Movie table	19
Figure 38: Insertion of data in 'Theater_Movie_User' table	19
Figure 39: Insertion of data	20
Figure 40: Outout of 'Theater_Movie_User'	20
Figure 41: Inserting data into 'HTMU' table	20
Figure 42: Insert data into 'HTMU' table	21
Figure 43: Output of 'HTMU' table	21
Figure 44: Inserting data into 'SHTMU' table	22
Figure 45: Output of 'SHTMU' table	22

Figure 46: Inserting data into 'TSHTMU' table.....	22
Figure 47: Output of 'TSHTMU' table	22

1 Introduction

This project aims to develop an efficient and well-structured database system for KumariCinemas, a cinema chain in Nepal, to enhance operational efficiency, improve customer experience, and streamline ticket management. The existing system faces several challenges, including inconsistent information about theater locations, hall numbers, seating capacities, and ticket pricing. Customers have also reported difficulties with booking tickets, cancellations, and the overall cinema experience.

To address these issues, this milestone focuses on analyzing and modeling the data, performing normalization, and designing an Entity-Relationship Diagram (ERD). A properly organized database is crucial to support key functions such as automated ticket booking, cancellation management, dynamic pricing during holidays or new movie releases, and accurate allocation of seats.

In this first milestone, the project examines the raw data, identifies all relevant attributes, keys, and dependencies, and applies normalization to transform the unstructured data into first, second, and third normal forms. This ensures that repeating groups, partial dependencies, and transitive dependencies are correctly resolved, producing a set of well-defined tables. An initial ERD is then created to represent entities, attributes, primary and foreign keys, and the relationships between them, which is further refined to produce the final ERD using Oracle SQL Developer Data Modeler.

The milestone also includes implementing the database by generating the DDL scripts from the ERD and creating the tables in Oracle, followed by populating them with sample data to verify correctness and functionality. Completing this milestone provides a solid foundation for the subsequent development of the web-based application, ensuring that KumariCinemas can efficiently manage shows, tickets, theaters, and customer bookings, while laying the groundwork for a scalable and user-friendly system.

2 Data Modelling and Normalization

2.1 Normalization

UNF (Unnormalized Form)

RD{RG}

UserId, Username, Address, {MovieId, Title, Duration, Language, Genre, ReleaseDate, {TheatreId, TheatreName, TheatreCity, {HallId, HallName, HallCapacity, {ShowId, ShowDate, ShowTimes, {TicketId, TicketPrice} }}}})

1NF

User(UserId, Username, Address)

Movie(MovieId, Title, Duration, Language, Genre, ReleaseDate, UserId*)

Theatre(TheatreId, TheatreName, TheatreCity, MovieId*, UserId*)

Hall(HallId, HallName, HallCapacity, TheatreId*, MovieId*, UserId*)

Show(ShowId, ShowDate, ShowTimes, HallId*, TheatreId*, MovieId*, UserId*)

Ticket(TicketId, TicketPrice, HallId*, TheatreId*, MovieId*, UserId*)

2NF

Removing partial dependencies:

User(UserId, Username, Address)

Movie(MovieId, Title, Duration, Language, Genre, ReleaseDate)

User_Movie(UserId*, MovieId*)

Theatre(TheatreId, TheatreName, TheatreCity)

Theatre_Movie_User(TheatreId*, MovieId*, UserId*)

Hall(HallId, HallName, HallCapacity)

Hall_Theatre_Movie_User(HallId*, TheatreId*, MovieId*, UserId*)

Show(ShowId, ShowDate, ShowTimes)

Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

Ticket(TicketId, TicketPrice)

Ticket_Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

3NF

$K \rightarrow NK \rightarrow NK$ (Removing transitive dependencies)

User(UserId, Username, Address)

Movie(MovieId, Title, Duration, Language, Genre, ReleaseDate)

User_Movie(UserId*, MovieId*)

Theatre(TheatreId, TheatreName, TheatreCity)

Theatre_Movie_User(TheatreId*, MovieId*, UserId*)

Hall(HallId, HallName, HallCapacity)

Hall_Theatre_Movie_User(HallId*, TheatreId*, MovieId*, UserId*)

Show(ShowId, ShowDate, ShowTime)

Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

Ticket(TicketId, TicketPrice)

Ticket_Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

2.2 Initial ERD Creation

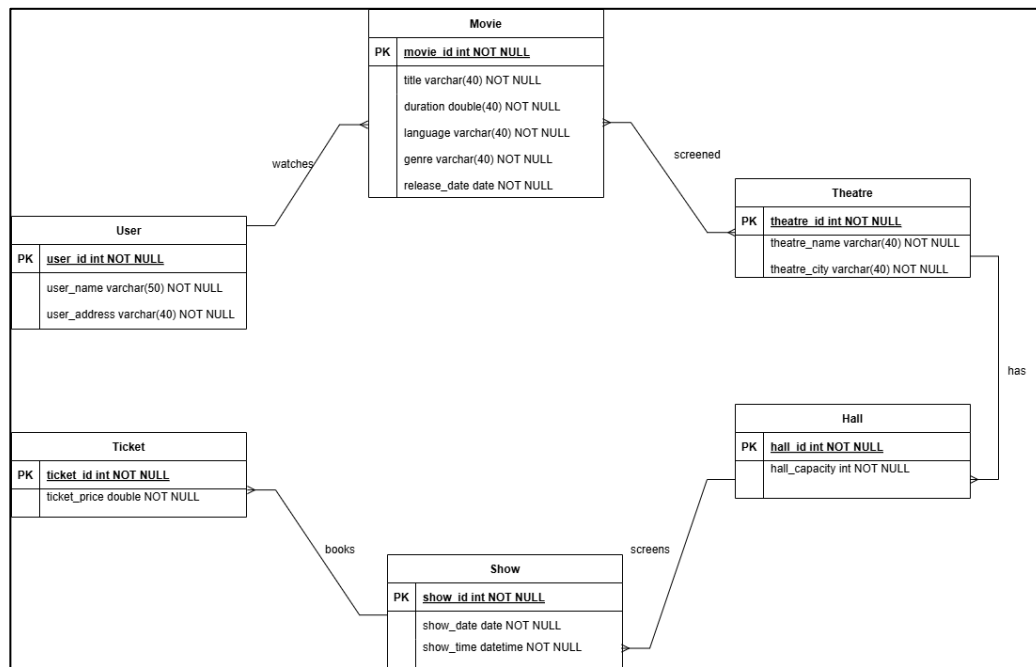


Figure 1: Initial ERD diagram

2.3 Final ERD Using Data Modeler

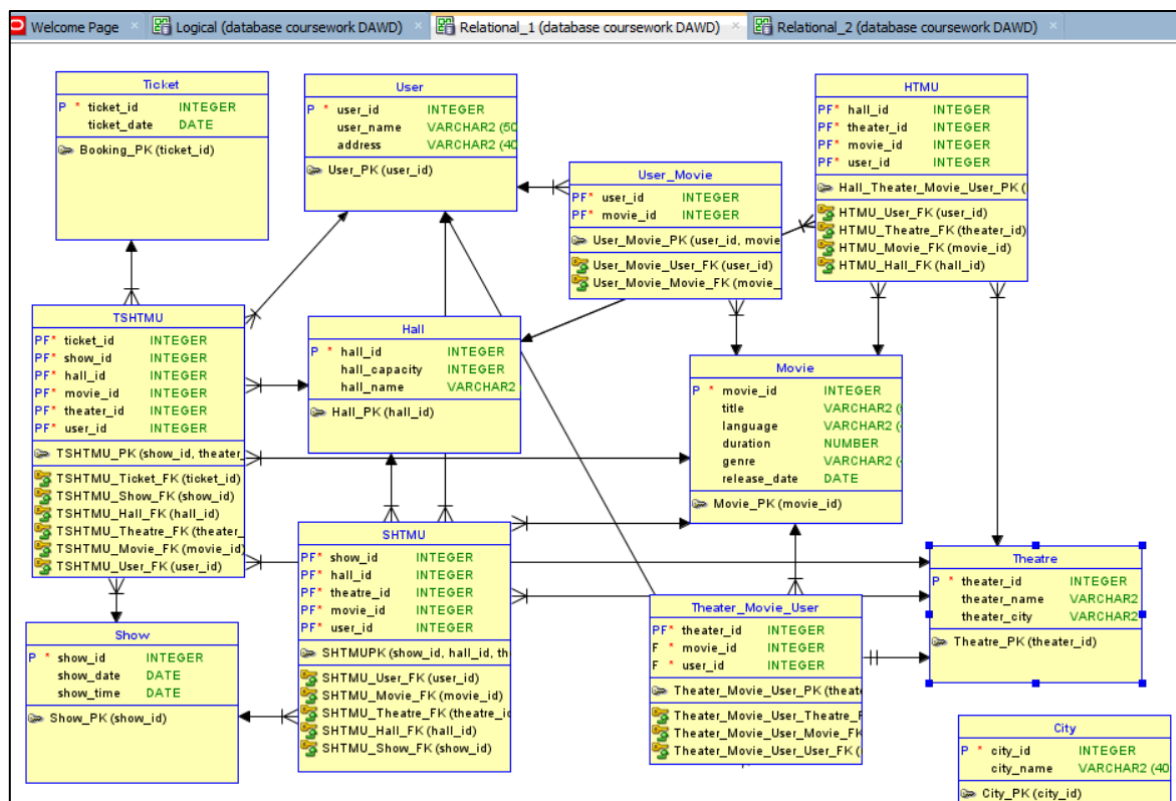


Figure 2: Final ERD using Data Modeler

2.4 Database implementation

2.4.1 Table Creation

```
SQL> CREATE TABLE "User"
2      (
3      user_id  INTEGER NOT NULL ,
4      user_name VARCHAR2 (50) ,
5      address  VARCHAR2 (40)
6      )
7  ;

Table created.

SQL> ALTER TABLE "User"
2      ADD CONSTRAINT User_PK PRIMARY KEY ( user_id ) ;

Table altered.
```

Figure 3: 'User' table creation query

```
SQL> CREATE TABLE Movie
2      (
3      movie_id  INTEGER NOT NULL ,
4      title     VARCHAR2 (50) ,
5      language  VARCHAR2 (40) ,
6      duration  NUMBER ,
7      genre     VARCHAR2 (40) ,
8      release_date DATE
9      )
10 ;

Table created.

SQL> ALTER TABLE Movie
2      ADD CONSTRAINT Movie_PK PRIMARY KEY ( movie_id ) ;

Table altered.
```

Figure 4: 'Movie' table creation query

```
SQL> CREATE TABLE Theatre
2      (
3      theater_id  INTEGER NOT NULL ,
4      theater_name VARCHAR2 (40) ,
5      city_id     INTEGER NOT NULL
6      )
7  ;

Table created.

SQL> ALTER TABLE Theatre
2      ADD CONSTRAINT Theatre_PK PRIMARY KEY ( theater_id ) ;

Table altered.
```

Figure 5: 'Theatre' table creation query

```
SQL> ALTER TABLE Theatre
  2  RENAME COLUMN city_id TO theater_city;

Table altered.
```

Figure 6: Theatre table altered

```
SQL> ALTER TABLE Theatre
  2  MODIFY theater_city VARCHAR2(30);
```

Figure 7: Theatre table altered

```
SQL> CREATE TABLE Hall
  2  (
  3      hall_id          INTEGER NOT NULL ,
  4      hall_capacity    INTEGER ,
  5      hall_name        VARCHAR2 (40)
  6  )
  7  ;

Table created.

SQL> ALTER TABLE Hall
  2      ADD CONSTRAINT Hall_PK PRIMARY KEY ( hall_id ) ;

Table altered.
```

Figure 8: 'Hall' table creation query

```
SQL> CREATE TABLE Show
  2  (
  3      show_id          INTEGER NOT NULL ,
  4      show_date        DATE ,
  5      show_time        DATE ,
  6      ticket_price    NUMBER
  7  )
  8  ;

Table created.

SQL> ALTER TABLE Show
  2      ADD CONSTRAINT Show_PK PRIMARY KEY ( show_id ) ;

Table altered.
```

Figure 9: 'Show' table creation query query

```
SQL> CREATE TABLE Ticket
2   (
3     ticket_id  INTEGER NOT NULL ,
4     ticket_date DATE
5   )
6   ;

Table created.

SQL>
SQL> ALTER TABLE Ticket
2     ADD CONSTRAINT Booking_PK PRIMARY KEY ( ticket_id ) ;

Table altered.
```

Figure 10: 'Ticket' table creation query

```
SQL> CREATE TABLE User_Movie
2   (
3     user_id  INTEGER NOT NULL ,
4     movie_id INTEGER NOT NULL
5   )
6   ;

Table created.

SQL>
SQL> ALTER TABLE User_Movie
2     ADD CONSTRAINT User_Movie_PK PRIMARY KEY ( user_id, movie_id ) ;

Table altered.

SQL>
SQL> ALTER TABLE User_Movie
2     ADD CONSTRAINT User_Movie_Movie_FK FOREIGN KEY
3     (
4       movie_id
5     )
6     REFERENCES Movie
7     (
8       movie_id
9     )
10    ;

Table altered.

SQL>
SQL> ALTER TABLE User_Movie
2     ADD CONSTRAINT User_Movie_User_FK FOREIGN KEY
3     (
4       user_id
5     )
6     REFERENCES "User"
7     (
8       user_id
9     )
10    ;

Table altered.
```

Figure 11: 'User_Movie' table creation query

```

SQL>
SQL> CREATE TABLE Theater_Movie_User
2      (
3          theater_id INTEGER NOT NULL ,
4          movie_id   INTEGER NOT NULL ,
5          user_id    INTEGER NOT NULL
6      )
7      ;

Table created.

SQL>
SQL> ALTER TABLE Theater_Movie_User
2      ADD CONSTRAINT Theater_Movie_User_PK PRIMARY KEY ( theater_id ) ;

Table altered.

SQL>
SQL> ALTER TABLE Theater_Movie_User
2      ADD CONSTRAINT Theater_Movie_User_Movie_FK FOREIGN KEY
3      (
4          movie_id
5      )
6      REFERENCES Movie
7      (
8          movie_id
9      )
10     ;

Table altered.

SQL>
SQL> ALTER TABLE Theater_Movie_User
2      ADD CONSTRAINT Theater_Movie_User_Theatre_FK FOREIGN KEY
3      (
4          theater_id
5      )
6      REFERENCES Theatre
7      (
8          theater_id
9      )
10     ;

Table altered.

```

Figure 12: 'Theater_Movie_User' table creation

```

SQL>
SQL> ALTER TABLE Theater_Movie_User
2      ADD CONSTRAINT Theater_Movie_User_User_FK FOREIGN KEY
3      (
4          user_id
5      )
6      REFERENCES "User"
7      (
8          user_id
9      )
10     ;

Table altered.

SQL> CREATE TABLE Hall_Theater_Movie_User
2      (
3          hall_id   INTEGER NOT NULL ,
4          theater_id INTEGER NOT NULL ,
5          movie_id  INTEGER NOT NULL ,
6          user_id   INTEGER NOT NULL
7      )
8      ;

Table created.

SQL>
SQL> ALTER TABLE Hall_Theater_Movie_User
2      ADD CONSTRAINT Hall_Theater_Movie_User_PK PRIMARY KEY ( hall_id, theater_id, movie_id, user_id ) ;

Table altered.

```

Figure 13: 'Theater_Movie_User' table alteration query

```

SQL> CREATE TABLE HTMU
2   (
3     hall_id    INTEGER NOT NULL ,
4     theater_id INTEGER NOT NULL ,
5     movie_id   INTEGER NOT NULL ,
6     user_id    INTEGER NOT NULL
7   )
8   ;

Table created.

SQL>
SQL> ALTER TABLE HTMU
2   ADD CONSTRAINT Hall_Theater_Movie_User_PK PRIMARY KEY ( hall_id, theater_id, movie_id, user_id ) ;

Table altered.

SQL>
SQL> ALTER TABLE HTMU
2   ADD CONSTRAINT HTMU_Hall_FK FOREIGN KEY
3   (
4     hall_id
5   )
6   REFERENCES Hall
7   (
8     hall_id
9   )
10  ;

Table altered.

SQL>
SQL> ALTER TABLE HTMU
2   ADD CONSTRAINT HTMU_Movie_FK FOREIGN KEY
3   (
4     movie_id
5   )
6   REFERENCES Movie
7   (
8     movie_id
9   )
10  ;

Table altered.

```

Figure 14: 'HTMU' table creation and alteration query

```

SQL> ALTER TABLE HTMU
2   ADD CONSTRAINT HTMU_Theatre_FK FOREIGN KEY
3   (
4     theater_id
5   )
6   REFERENCES Theatre
7   (
8     theater_id
9   )
10  ;

Table altered.

SQL>
SQL> ALTER TABLE HTMU
2   ADD CONSTRAINT HTMU_User_FK FOREIGN KEY
3   (
4     user_id
5   )
6   REFERENCES "User"
7   (
8     user_id
9   )
10  ;

Table altered.

```

Figure 15: 'HTMU' table alteration query

```
SQL> CREATE TABLE SHTMU
2      (
3          show_id    INTEGER NOT NULL ,
4          hall_id    INTEGER NOT NULL ,
5          theatre_id INTEGER NOT NULL ,
6          movie_id   INTEGER NOT NULL ,
7          user_id    INTEGER NOT NULL
8      )
9      ;

Table created.
```

Figure 16: 'SHTMU' table creation query

```
SQL>
SQL> ALTER TABLE SHTMU
2      ADD CONSTRAINT SHTMU_Hall_FK FOREIGN KEY
3      (
4          hall_id
5      )
6      REFERENCES Hall
7      (
8          hall_id
9      )
10     ;

Table altered.

SQL>
SQL> ALTER TABLE SHTMU
2      ADD CONSTRAINT SHTMU_Movie_FK FOREIGN KEY
3      (
4          movie_id
5      )
6      REFERENCES Movie
7      (
8          movie_id
9      )
10     ;

Table altered.

SQL>
SQL> ALTER TABLE SHTMU
2      ADD CONSTRAINT SHTMU_Show_FK FOREIGN KEY
3      (
4          show_id
5      )
6      REFERENCES Show
7      (
8          show_id
9      )
10     ;

Table altered.
```

Figure 17: 'SHTMU' table alteration query

```
SQL>
SQL> ALTER TABLE SHTMU
2   ADD CONSTRAINT SHTMU_Theatre_FK FOREIGN KEY
3   (
4     theatre_id
5   )
6   REFERENCES Theatre
7   (
8     theater_id
9   )
10  ;

Table altered.

SQL>
SQL> ALTER TABLE SHTMU
2   ADD CONSTRAINT SHTMU_User_FK FOREIGN KEY
3   (
4     user_id
5   )
6   REFERENCES "User"
7   (
8     user_id
9   )
10  ;

Table altered.

SQL> ALTER TABLE SHTMU
2   ADD CONSTRAINT SHTMUPK PRIMARY KEY ( show_id, hall_id, theatre_id, movie_id, user_id ) ;

Table altered.
```

Figure 18: 'SHTMU' table alteration query

```
SQL> ALTER TABLE TSHTMU
2   ADD CONSTRAINT TSHTMU_Hall_FK FOREIGN KEY
3   (
4     hall_id
5   )
6   REFERENCES Hall
7   (
8     hall_id
9   )
10  ;

Table altered.

SQL>
SQL> ALTER TABLE TSHTMU
2   ADD CONSTRAINT TSHTMU_Movie_FK FOREIGN KEY
3   (
4     movie_id
5   )
6   REFERENCES Movie
7   (
8     movie_id
9   )
10  ;

Table altered.

SQL>
SQL> ALTER TABLE TSHTMU
2   ADD CONSTRAINT TSHTMU_Show_FK FOREIGN KEY
3   (
4     show_id
5   )
6   REFERENCES Show
7   (
8     show_id
9   )
10  ;

Table altered.
```

Figure 19: 'TSHTMU' table creation query

```
SQL>
SQL> ALTER TABLE TSHTMU
 2      ADD CONSTRAINT TSHTMU_Theatre_FK FOREIGN KEY
 3      (
 4        theater_id
 5      )
 6      REFERENCES Theatre
 7      (
 8        theater_id
 9      )
10 ;

Table altered.

SQL>
SQL> ALTER TABLE TSHTMU
 2      ADD CONSTRAINT TSHTMU_Ticket_FK FOREIGN KEY
 3      (
 4        ticket_id
 5      )
 6      REFERENCES Ticket
 7      (
 8        ticket_id
 9      )
10 ;

Table altered.

SQL>
SQL> ALTER TABLE TSHTMU
 2      ADD CONSTRAINT TSHTMU_User_FK FOREIGN KEY
 3      (
 4        user_id
 5      )
 6      REFERENCES "User"
 7      (
 8        user_id
 9      )
10 ;

Table altered.
```

Figure 20: 'TSHTMU' table alteration query

2.4.2 Table Insertion

```
SQL> INSERT INTO USERS(user_id,user_name,address)
 2  VALUES(1,'Simanka','Thaiba');

1 row created.

SQL> INSERT INTO USERS(user_id,user_name,address)
 2  VALUES(2,'Riwaj','Balaju');

1 row created.

SQL> INSERT INTO USERS(user_id,user_name,address)
 2  VALUES(3,'Aasra','NewRoad');

1 row created.
```

Figure 21: Inserting data into 'USERS' table


```

SQL> INSERT INTO USERS(user_id,user_name,address)
  2  VALUES(4,'Simran','Thamel');

1 row created.

SQL> INSERT INTO USERS(user_id,user_name,address)
  2  VALUES(5,'Sam','Asan');

1 row created.

```

Figure 22: Inserting data into 'USERS' table

- Output:

```

SQL> SELECT * FROM USERS;

```

USER_ID	USER_NAME	ADDRESS
1	Simanka	Thaiba
2	Riwaj	Balaju
3	Aasra	NewRoad
4	Simran	Thamel
5	Sam	Asan

Figure 23: Output of 'USERS' table

- Movie Table

```

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
  2  VALUES(1, 'Mean girls', 1.5 , 'Rom-Com',TO_DATE('2002-01-05', 'YYYY-MM-DD'))
  3  ;

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
  2  VALUES(2, 'It Ends with us', 2.5 , 'Romantic-Drama',TO_DATE('2026-01-15', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
  2  VALUES(3, 'NoteBook', 2 , 'Rom-Com',TO_DATE('2026-01-08', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
  2  VALUES(4, 'Batman', 2.5 , 'Action',TO_DATE('2026-01-04', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
  2  VALUES(5, 'Harry Potter', 2 , 'Fantasy',TO_DATE('2026-01-10', 'YYYY-MM-DD'));

1 row created.

```

Figure 24: Insertion of data in 'Movie' table

```

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 1;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 2;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 3;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 4;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 5;

1 row updated.

```

Figure 25: Update on 'Movie' table

- Output

```
SQL> select * from Movie;
```

MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_D
1	Mean girls	English	1.5	Rom-Com	05-JAN-02
2	It Ends with us	English	2.5	Romantic-Drama	15-JAN-26
3	NoteBook	English	2	Rom-Com	08-JAN-26
4	Batman	English	2.5	Action	04-JAN-26
5	Harry Potter	English	2	Fantasy	10-JAN-26

Figure 26: Output of Movie table

- Theater Table

```

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
  2  VALUES(1,'Kumari Complex','KamalPokari');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
  2  VALUES(2,'QFX','Pulchowk');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
  2  VALUES(3,'One Cinemas','Kalimati');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
  2  VALUES(4,'One Cinemas','Baneshwor');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
  2  VALUES(5,'BishwoJyoti Cineplex','Jamal');

1 row created.

SQL> COMMIT;

Commit complete.

```

Figure 27: Insertion of data in 'Theatre' table

- Output

```

SQL> select * from Theatre;

```

THEATER_ID	THEATER_NAME	THEATER_CITY
1	Kumari Complex	KamalPokari
2	QFX	Pulchowk
3	One Cinemas	Kalimati
4	One Cinemas	Baneshwor
5	BishwoJyoti Cineplex	Jamal

Figure 28: Output of Theatre table

- Hall Table

```

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(1, 'A', 60);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(2, 'B', 80);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(3, 'C', 100);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(4, 'D', 110);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(5, 'E', 120);

1 row created.

SQL> COMMIT;

Commit complete.

```

Figure 29: Insertion of data in "Hall" table

- Output

```

SQL> select * from hall;

```

HALL_ID	HALL_CAPACITY	HALL_NAME
1	60	A
2	80	B
3	100	C
4	110	D
5	120	E

Figure 30: Output of 'Hall' table

- Show table

```

SQL> INSERT into Show(show_id,show_date,show_time)
  2  VALUES(1,TO_DATE('2004-06-07', 'YYYY-MM-DD'),TO_TIMESTAMP('2004-06-07 18:30:00', 'YYYY-MM-DD HH24:MI:SS'));

1 row created.

```

Figure 31: Inserting data into 'Show' table

```

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(2,TO_DATE('2026-01-15', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-15 18:30:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(3,TO_DATE('2026-01-08', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-08 13:00:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(4,TO_DATE('2026-01-04', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-04 09:30:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(5,TO_DATE('2026-01-10', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-10 10:00:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

```

Figure 32: Inserting of data in Show table

- Output

```

SQL> select * from Show;

  SHOW_ID  SHOW_DATE  SHOW_TIME
-----
         1  07-JUN-04  07-JUN-04
         2  15-JAN-26  15-JAN-26
         3  08-JAN-26  08-JAN-26
         4  04-JAN-26  04-JAN-26
         5  10-JAN-26  10-JAN-26

```

Figure 33: Output of 'Show' table

- Ticket table

```

SQL> INSERT into Ticket(ticket_id,ticket_date)
      2 VALUES(1,TO_DATE('2026-01-05', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
      2 VALUES(2,TO_DATE('2026-01-15', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
      2 VALUES(3,TO_DATE('2026-01-08', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
      2 VALUES(4,TO_DATE('2026-01-04', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
      2 VALUES(5,TO_DATE('2026-01-10', 'YYYY-MM-DD'));

1 row created.

```

Figure 34: Inserting of data in Ticket table

- Output

TICKET_ID	TICKET_DA
1	05-JAN-26
2	15-JAN-26
3	08-JAN-26
4	04-JAN-26
5	10-JAN-26

Figure 35: Output of 'Ticket' table

- User_Movie Table

```

SQL> INSERT INTO User_Movie(movie_id,user_id)
      2 VALUES(1,1);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
      2 VALUES(1,2);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
      2 VALUES(3,4);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
      2 VALUES(4,5);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
      2 VALUES(2,3);

1 row created.

```

Figure 36: Inserting data into 'User_Movie' table

- Output

```

SQL> select * from User_Movie;

  USER_ID  MOVIE_ID
-----
         1         1
         2         1
         3         2
         4         3
         5         4

```

Figure 37: Output of User_Movie table

- Theater_Movie_User table

```

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
      2 VALUES(1,1,2);

1 row created.

```

Figure 38: Insertion of data in 'Theater_Movie_User' table

```

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(2,2,3);

1 row created.

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(3,4,5);

1 row created.

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(5,4,2);

1 row created.

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(4,1,1);

1 row created.

```

Figure 39: Insertion of data

- Output

```

SQL> select * from Theater_Movie_User;

THEATER_ID  MOVIE_ID  USER_ID
-----
          1          1          2
          2          2          3
          3          4          5
          5          4          2
          4          1          1

```

Figure 40: Outout of 'Theater_Movie_User'

- HTMU table

```

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2 VALUES(2,5,5,4);

1 row created.

```

Figure 41: Inserting data into 'HTMU' table


```

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2 VALUES(3,3,5,1);

1 row created.

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2 VALUES(4,2,3,1);

1 row created.

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2 VALUES(5,2,3,4);

1 row created.

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2 VALUES(1,4,3,2);

1 row created.

```

Figure 42: Insert data into 'HTMU' table

- Output

```

SQL> select * from HTMU;

  HALL_ID THEATER_ID  MOVIE_ID  USER_ID
-----
        1         4         3         2
        2         5         5         4
        3         3         5         1
        4         2         3         1
        5         2         3         4

```

Figure 43: Output of 'HTMU' table

- SHTMU table

```

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2 VALUES(5,1,1,2,2);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2 VALUES(4,3,1,3,4);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2 VALUES(3,2,1,5,4);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2 VALUES(2,2,3,4,4);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2 VALUES(1,2,5,5,4);

1 row created.

```

Figure 44: Inserting data into 'SHTMU' table

- Output

```
SQL> select * from SHTMU;
```

SHOW_ID	HALL_ID	THEATRE_ID	MOVIE_ID	USER_ID
1	2	5	5	4
2	2	3	4	4
3	2	1	5	4
4	3	1	3	4
5	1	1	2	2

Figure 45: Output of 'SHTMU' table

- TSHTMU table

```
SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(1,1,1,2,2,2)
3 ;

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(2,2,3,3,4,4);

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(3,3,3,4,4,4);

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(4,5,1,2,3,4);

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(5,5,1,1,3,3);

1 row created.
```

Figure 46: Inserting data into 'TSHTMU' table

- Output

```
SQL> select * from TSHTMU;
```

TICKET_ID	SHOW_ID	HALL_ID	MOVIE_ID	THEATER_ID	USER_ID
1	1	1	2	2	2
2	2	3	4	3	4
3	3	3	4	4	4
5	5	1	3	1	3
4	5	1	3	2	4

Figure 47: Output of 'TSHTMU' table

